TẬP PHỔ BIẾN & LUẬT KẾT HỢP

A. MỤC TIÊU CHƯƠNG

Hướng dẫn sinh viên triển khai thực hiện việc tìm tập phổ biến bằng giải thuật Apriori.

Hướng dẫn sinh viên trích xuất luật kết hợp từ tập phổ biến và cách thức trình bày một luật kết hợp

Hướng dẫn sinh viên ôn tập lý thuyết về tập phổ biến và luật kết hợp

Hướng dẫn sinh viên cách tiếp cận để tìm kiếm các đoạn mã nguồn bằng Python hỗ trợ cho việc xây dựng các giải thuật

B. KẾT CẤU CHƯƠNG

Chương bao gồm 2

- Xác định tập phổ biến với giải thuật Apriori

- Xác định luật kết hợp từ tập phổ biến

C. NỘI DUNG CHƯƠNG

4.1. XÁC ĐỊNH TẬP PHỔ BIẾN VỚI GIẢI THUẬT APRIORI

4.1.1. Ôn tập lý thuyết với GPT

1) Giải thuật Apriori hoạt động như thế nào? Hãy giải thích các bước chính trong quá trình tìm tập phổ biến

- Nguyên lý cốt lõi của Apriori: "Nếu một tập hợp items nào đó phổ biến → thì mọi tập con của nó cũng phải phổ biến". (Ngược lại: nếu một tập con không phổ biến → toàn bộ tập lớn hơn chứa nó chắc chắn không phổ biến → loại bỏ luôn!)

- Mục tiêu: Tìm tất cả các tập phổ biến (frequent itemsets) có support ≥ min_support.

- Các bước chính của thuật toán Apriori (rất rõ ràng):

+ Bước 1: Tạo L_1 (tập 1-item phổ biến)

- Đếm số lần xuất hiện của từng món hàng riêng lẻ.
- Loại bỏ những món có support < min_support. → Kết quả: danh sách các món hàng phổ biến (L_1).

+ Bước 2: Tạo các tập ứng viên C_k (k-itemsets)

- Từ L_{k-1} (các tập phổ biến kích thước k-1) → sinh ra các tập ứng viên kích thước k bằng cách ghép đôi (join).
- Ví dụ: từ {bánh mì}, {bơ} → sinh {bánh mì, bơ}.

+ Bước 3: Cắt tỉa (pruning) ngay lập tức

- Kiểm tra mọi tập con kích thước (k-1) của tập ứng viên.
- Nếu có dù chỉ 1 tập con không nằm trong L_{k-1} → loại bỏ tập ứng viên đó luôn (vì chắc chắn không phổ biến).

+ Bước 4: Đếm support thực tế bằng cách quét lại toàn bộ dữ liệu giao dịch để đếm support của những tập ứng viên còn lại sau khi cắt tỉa.

SGU25

+ Bước 5: Lọc ra L_k bằng cách giữ lại những tập ứng viên có support ≥ min_support → thành L_k (tập phổ biến kích thước k).

+ Lặp lại từ bước 2 → 5 bằng cách tăng k lên 1 (k=2,3,4…) cho đến khi: Không còn sinh được tập ứng viên nào nữa, hoặc L_k rỗng.

+ Kết thúc → Gộp tất cả $L_1 \cup L_2 \cup L_3 \ldots$ = tất cả tập phổ biến trong dữ liệu.

2) Các tham số như support, confidence, và lift có ý nghĩa gì trong khai thác luật kết hợp? Chúng được sử dụng như thế nào trong Apriori?

Tham số	Ý nghĩa	Công thức	Dùng trong Apriori để làm gì?	Giá trị thường dùng
Support	% giao dịch chứa tập hợp đó	$Support(X) = (số\ giao\ dịch\ chứa\ X)/tổng$	Lọc tập phổ biến (bước chính)	0.01 – 0.1 (1%–10%)
Confidence	Khi mua X thì mua Y bao nhiêu %?	$Conf(X \to Y) = Support(X \cup Y)/Support(X)$	Lọc luật mạnh sau khi có tập phổ biến	≥ 0.5 (50%)
Lift	Quy tắc mạnh hơn ngẫu nhiên bao nhiêu lần?	$Lift(X \to Y) = Conf(X \to Y)/Support(Y)$	Loại luật vô nghĩa dù confidence cao	≥ 3 (mạnh), >1 là tốt

3) Sự khác biệt giữa giải thuật Apriori và các thuật toán khác như FP-Growth trong việc tìm tập phổ biến là gì?

Tiêu chí	Apriori	FP-Growth
Cấu trúc dữ liệu	Không dùng cấu trúc đặc biệt	Dùng FP-Tree (cây nén rất gọn)
Số lần quét database	Nhiều lần (mỗi lần sinh Lk quét 1 lần)	Chỉ quét database 2 lần
Tạo tập ứng viên	Có (C_2, C_3, … rất nhiều)	Không tạo tập ứng viên nào
Chi phí bộ nhớ	Thấp với tập ứng viên lớn → chậm	Cao hơn (xây cây), nhưng tổng thể nhanh hơn
Tốc độ	Chậm khi dữ liệu lớn hoặc min_support thấp	Nhanh hơn rất nhiều (thường 10–100 lần)
Phù hợp với dữ liệu	Dữ liệu thưa, ít giao dịch	Dữ liệu dày đặc hoặc rất lớn
Độ phức tạp	Dễ hiểu, dễ cài đặt	Phức tạp hơn nhưng hiệu quả vượt trội
Thư viện Python	apyori, mlxtend	mlxtend, pyspark
Kết luận	Dùng để học, demo, dữ liệu nhỏ	Dùng thực tế cho dataset lớn, hiệu năng cao

4) Viết đoạn code mẫu bằng Python (sử dụng thư viện như **mlxtend** hoặc **apyori**) để triển khai giải thuật Apriori không? Hãy mô tả các bước thực hiện

- Mã mẫu trong Python sử dụng thư viện mlxtend để triển khai giải thuật Apriori:

```
# pip install mlxtend pandas
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
# Tạo dữ liệu giỏ hàng mẫu
data = [['sữa', 'bánh mì', 'bơ'],
        ['bia', 'bim bim', 'đậu phộng'],
        ['sữa', 'bánh mì', 'bơ', 'mứt'],
        ['bánh mì', 'bơ'],
        ['sữa', 'bánh mì', 'bơ', 'bia'],
        ['bim bim', 'đậu phộng']]
# Chuyển sang dạng One-hot encoding
from mlxtend.preprocessing import TransactionEncoder
te = TransactionEncoder()
te_ary = te.fit(data).transform(data)
df = pd.DataFrame(te_ary, columns=te.columns_)
# Chạy Apriori tìm tập phổ biến
frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)  # 40% giao
dịch
print("Tập phổ biến:")
print(frequent_itemsets)
# Tạo luật kết hợp
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)
print("\nLuật kết hợp:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
# Lọc luật hay (ví dụ lift > 1.2 và confidence > 0.7)
good_rules = rules[(rules['lift'] > 1.2) & (rules['confidence'] > 0.7)]
print("\nLuật hay nhất:")
print(good_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

- Bảng mô tả các bước thực hiện:

Bước	Nội dung	Code chính
1	Chuẩn bị dữ liệu dạng list giao dịch	data = [['sữa', 'bánh mì'], ...]
2	One-hot encoding	TransactionEncoder()
3	Chạy Apriori tìm tập phổ biến	apriori(..., min_support=0.x)
4	Sinh luật kết hợp	association_rules(..., metric="lift")
5	Lọc luật hay	rules[rules['lift'] > 1 & rules['confidence'] > 0.7]

5) Làm thế nào để tiền xử lý dữ liệu giao dịch (transactional data) trong Python trước khi áp dụng Apriori?

- Để có thể thực hiện tiền xử lý dữ liệu giao dịch trong Python trước khi áp dụng Apriori ta làm các bước:

Bước	Mục đích	Code mẫu (mlxtend)	Ghi chú quan trọng
1	Đọc dữ liệu	df = pd.read_csv('data.csv', header=None)	Thường không có header, mỗi dòng = 1 giao dịch
2	Xử lý giá trị trống (NaN)	df = df.fillna('') hoặc df = df.where(pd.notnull(df), None)	Không để NaN trong danh sách items
3	Chuyển thành danh sách giao dịch	transactions = [] for i in range(len(df)): trans = [str(item) for item in df.iloc[i] if pd.notnull(item)] transactions.append(trans)	Loại bỏ NaN và chuyển thành string (tránh lỗi kiểu)
4	One-hot encoding (bắt buộc cho mlxtend)	from mlxtend.preprocessing import TransactionEncoder te = TransactionEncoder() te_ary = te.fit(transactions).transform(transactions) basket = pd.DataFrame(te_ary, columns=te.columns_)	Đây là dạng mà apriori của mlxtend nhận
5	(Tùy chọn) Loại bỏ items quá hiếm	item_counts = basket.sum() basket = basket.loc[:, item_counts[item_counts >= 5].index]	Tăng tốc độ, giảm bộ nhớ
6	Sẵn sàng chạy Apriori	frequent_itemsets = apriori(basket, min_support=0.01, use_colnames=True)	Kết thúc

6) Hàm nào trong Python để tính toán tập phổ biến và luật kết hợp từ giải thuật Apriori? Hãy chia sẻ một đoạn code mẫu

- Bảng biểu diễn hàm tính toán tập phổ biến và luật kết hợp từ giải thuật Apriori:

Nhiệm vụ	Hàm trong Python (mlxtend)	Thư viện cần cài
Tìm tập phổ biến	apriori()	from mlxtend.frequent_patterns import apriori
Tạo luật kết hợp	association_rules()	from mlxtend.frequent_patterns import association_rules

- Mã mẫu trong Python:

```
# pip install mlxtend pandas
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
# 1. Dữ liệu mẫu
transactions = [
    ['sữa', 'bánh mì', 'bơ'],
    ['bia', 'bim bim'],
    ['sữa', 'bánh mì', 'bơ', 'mứt'],
    ['bánh mì', 'bơ'],
    ['sữa', 'bánh mì', 'bơ', 'bia']
]
# 2. One-hot encoding
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
```

```
df = pd.DataFrame(te_ary, columns=te.columns_)
# 3. Tính tập phổ biến (min_support = 0.5 → xuất hiện ít nhất 50% giao dịch)
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)
# 4. Tạo luật kết hợp
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)
# 5. In kết quả đẹp
print("Tập phổ biến:")
print(frequent_itemsets)
print("\nLuật kết hợp:")
print(rules[['antecedents','consequents','support','confidence','lift']])
```

7) Điều chỉnh ngưỡng support và confidence trong Apriori để đạt được kết quả tốt hơn không? Hãy mô tả cách thực hiện

- Cách điều chỉnh ngưỡng và mô tả các bước thực hiện:

Mục tiêu bạn muốn	**Cách điều chỉnh ngưỡng**	**Giá trị thường thử (dataset ~5000–10000 giao dịch)**	**Hậu quả nếu để quá cao/thấp**
Muốn nhiều luật hơn (để khám phá)	↓ min_support + ↓ min_confidence	min_support = 0.005–0.02 min_confidence = 0.2–0.4	Ra rất nhiều luật (có thể hàng nghìn), dễ bị nhiễu
Muốn ít luật nhưng chắc chắn, ý nghĩa cao (dùng thực tế kinh doanh)	↑ min_support + ↑ min_confidence + ↑ min_lift	min_support = 0.02–0.05 min_confidence = 0.5–0.8 min_lift ≥ 3	Chỉ còn vài chục luật cực mạnh, đáng tin để triển khai
Tìm các sản phẩm hiếm nhưng khi mua cùng thì rất mạnh	↓ min_support rất thấp ↑ min_confidence và min_lift cao	min_support = 0.001–0.005 min_confidence ≥ 0.6 min_lift ≥ 5	Phát hiện "viên ngọc ẩn" (ví dụ: mua máy ảnh → mua thẻ nhớ 512GB)
Tránh luật quá hiển nhiên (ví dụ: bánh mì → bơ quá phổ biến)	Thêm điều kiện min_lift cao hơn	min_lift ≥ 3–5	Loại bỏ các luật "ai cũng biết rồi"

8) Cách trực quan hóa các luật kết hợp (association rules) dưới dạng biểu đồ mạng (network graph) hoặc heatmap trong Python

- Cách trực quan hóa các luật kết hợp dưới dạng biểu đồ mạng trong Python:

```
# pip install networkx matplotlib seaborn
import matplotlib.pyplot as plt
import networkx as nx
from mlxtend.frequent_patterns import apriori, association_rules
# Giả sử bạn đã có 'rules' từ association_rules
# Ví dụ:
# rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)
plt.figure(figsize=(12,8))
G = nx.DiGraph()
for i in range(len(rules)):
    ante = ', '.join(list(rules.iloc[i]['antecedents']))
    cons = ', '.join(list(rules.iloc[i]['consequents']))
    G.add_edge(ante, cons,
               weight=rules.iloc[i]['lift'],

label=f"lift={rules.iloc[i]['lift']:.2f}\nconf={rules.iloc[i]['confidence']:.2f}")

pos = nx.spring_layout(G, k=0.8, iterations=50)
edges = G.edges()
```

```
weights = [G[u][v]['weight']*1.5 for u,v in edges]
nx.draw(G, pos, with_labels=True, node_color='lightblue',
        node_size=3000, font_size=10, font_weight='bold',
        edge_color='gray', width=weights, arrows=True, arrowsize=20,
        arrowstyle='->')
edge_labels = nx.get_edge_attributes(G,'label')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=9)
plt.title("Association Rules Network Graph (Arrow = mua → mua)", fontsize=14)
plt.axis('off')
plt.show()
```

- Cách trực quan hóa các luật kết hợp dưới dạng biểu đồ heatmap trong Python:

```
import seaborn as sns
import matplotlib.pyplot as plt
# Tạo bảng pivot: hàng = antecedents, cột = consequents, giá trị = lift
rules['antecedents'] = rules['antecedents'].apply(lambda x: ', '.join(list(x)))
rules['consequents'] = rules['consequents'].apply(lambda x: ', '.join(list(x)))
pivot = rules.pivot(index='antecedents', columns='consequents', values='lift')
plt.figure(figsize=(10,8))
sns.heatmap(pivot, annot=True, fmt=".2f", cmap="YlGnBu", linewidths=.5,
cbar_kws={'label': 'Lift'})
plt.title("Heatmap of Association Rules (Lift)", fontsize=14)
plt.ylabel("Nếu mua (antecedents)")
plt.xlabel("Thì mua (consequents)")
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

4.1.2. Bài làm mẫu

Bài toán 1: Tìm tập phổ biến trong dữ liệu giao dịch mua hàng tại siêu thị. Dữ liệu có thể lấy từ https://www.kaggle.com/code/mgmarques/customer-segmentation-and-market-basket-analysis

Nhiệm vụ 1: sử dụng thư viện mlxtend để tìm tập phổ biến có trong dữ liệu

1. Cài đặt các thư viện **mlxtend** và import các gói dữ liệu

```
# To install mlxtend, go to the Anaconda prompt and execute pip
install mlxtend
!pip install mlxtend
import matplotlib.pyplot as plt
import mlxtend.frequent_patterns
import mlxtend.preprocessing
import numpy
import pandas
```

2. Thực hiện một số tính toán để hiểu các khái niệm cơ bản như support, confident và lift

SGU25

```
# danh sách các giao dịch mua hàng
example = [
    ['milk', 'bread', 'apples', 'cereal', 'jelly', 'cookies', 'salad', 'tomatoes'],
    ['beer', 'milk', 'chips', 'salsa', 'grapes', 'wine', 'potatoes', 'eggs', 'carrots'],
    ['diapers', 'baby formula', 'milk', 'bread', 'chicken', 'asparagus', 'cookies'],
    ['milk', 'cookies', 'chicken', 'asparagus', 'broccoli', 'cereal', 'orange juice'],
    ['steak', 'asparagus', 'broccoli', 'chips', 'salsa', 'ketchup', 'potatoes', 'salad'],
    ['beer', 'salsa', 'asparagus', 'wine', 'cheese', 'crackers', 'strawberries', 'cookies'],
    ['chocolate cake', 'strawberries', 'wine', 'cheese', 'beer', 'milk', 'orange juice'],
    ['chicken', 'peas', 'broccoli', 'milk', 'bread', 'eggs', 'potatoes', 'ketchup', 'crackers'],
    ['eggs', 'bread', 'cheese', 'turkey', 'salad', 'tomatoes', 'wine', 'steak', 'carrots'],
    ['bread', 'milk', 'tomatoes', 'cereal', 'chicken', 'turkey', 'chips', 'salsa', 'diapers']
]
# Tính support
# the number of transactions
N = len(example)
# the frequency of milk
f_x = sum(['milk' in i for i in example])
# the frequency of bread
f_y = sum(['bread' in i for i in example])
# the frequency of milk and bread
f_x_y = sum([
    all(w in i for w in ['milk', 'bread'])
    for i in example
])
# support (supp)
support = f_x_y / N
print("Support = {}".format(round(support, 4))) # support = 0.4
# confidence: x -> y
confidence = support / (f_x / N)
print("Confidence = {}".format(round(confidence, 4)))# Confidence = 0.5714
# lift: x -> y
lift = confidence / (f_y / N)
```

SGU25

```
print("Lift = {}".format(round(lift, 4))) # Lift = 1.1429
# leverage: x -> y
leverage = support - ((f_x / N) * (f_y / N))
print("Leverage = {}".format(round(leverage, 4)))
# conviction: x -> y
conviction = (1 - (f_y / N)) / (1 - confidence)
print("Conviction = {}".format(round(conviction, 4)))
```

3. Nạp dữ liệu, làm sạch dữ liệu và biến đổi dữ liệu về định dạng phù hợp để thực hiện xây dựng mô hình (chuyển dữ liệu từ 1 hóa đơn có nhiều dòng thành 1 hóa đơn chỉ có 1 dòng chứa các sản phẩm)

```
online = pandas.read_excel( io="Online Retail.xlsx",
    sheet_name="Online Retail",
    header=0
)
# create new column called IsCPresent
online['IsCPresent'] = ( # looking for C in InvoiceNo column
    online['InvoiceNo'] # convert column to string type for the apply
                        function below
    .astype(str) # set element to 1 if C present otherwise 0
    .apply(lambda x: 1 if x.find('C') != -1 else 0)
)
online1 = (
    online # filter out non-positive quantity values
    .loc[online["Quantity"] > 0] # remove InvoiceNos starting with C
    .loc[online['IsCPresent'] != 1] # column filtering
    .loc[:, ["InvoiceNo", "Description"]]# dropping all rows with at
least
                                        one missing value
    .dropna()
)
# extract unique invoice numbers as list
invoice_no_list = online1.InvoiceNo.tolist()
invoice_no_list = list(set(invoice_no_list))
print("Length of list of invoice numbers:
              {ln}".format(ln=len(invoice_no_list)))

# get 5000 transaction
subset_invoice_no_list = invoice_no_list[0:5000]
# filter data set down to based on subset of invoice number list
online1 = online1.loc[online1["InvoiceNo"].isin(subset_invoice_no_list)]
```

SGU25

```
invoice_item_list = []
for num in list(set(online1.InvoiceNo.tolist())):
    # filter data set down to one invoice number
    tmp_df = online1.loc[online1['InvoiceNo'] == num]
    # extract item descriptions and convert to list
    tmp_items = tmp_df.Description.tolist()
    # append list invoice_item_list
    invoice_item_list.append(tmp_items)
print(invoice_item_list[1:5])
```

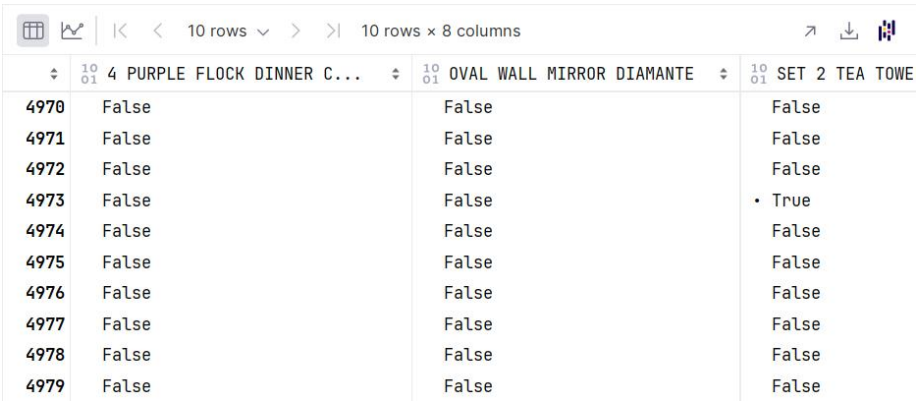
Kết quả thực hiện

```
[['HAND WARMER UNION JACK', 'HAND WARMER RED POLKA DOT'], ['ASSORTED
COLOUR BIRD ORNAMENT', "POPPY'S PLAYHOUSE BEDROOM ", "POPPY'S PLAYHOUSE
KITCHEN", 'FELTCRAFT PRINCESS CHARLOTTE DOLL', 'IVORY KNITTED MUG COSY ',
'BOX OF 6 ASSORTED COLOUR TEASPOONS', 'BOX OF VINTAGE JIGSAW BLOCKS ',
'BOX OF VINTAGE ALPHABET BLOCKS', 'HOME BUILDING BLOCK WORD', 'LOVE
BUILDING BLOCK WORD', 'RECIPE BOX WITH METAL HEART', 'DOORMAT NEW
ENGLAND'], ['JAM MAKING SET WITH JARS', 'RED COAT RACK PARIS FASHION',
'YELLOW COAT RACK PARIS FASHION', 'BLUE COAT RACK PARIS FASHION'], ['BATH
BUILDING BLOCK WORD']]
```

4. Mã hóa dữ liệu về dạng dữ liệu mà giải thuật apriori trong gói mlxtend yêu cầu

```
online_encoder = mlxtend.preprocessing.TransactionEncoder()
online_encoder_array = online_encoder.fit_transform(invoice_item_list)
online_encoder_df = pandas.DataFrame(
    online_encoder_array,
    columns=online_encoder.columns_
)
# this is a very big table, so for more
             easy viewing only a subset is printed
online_encoder_df.loc[
    4970:4979,
    online_encoder_df.columns.tolist()[0:8]
]
print("Data dimension (row count, col count):
                      {dim}".format(dim=online_encoder_df.shape))
```

Kết quả thực hiện



10 rows | 10 rows × 8 columns

	4 PURPLE FLOCK DINNER C...	OVAL WALL MIRROR DIAMANTE	SET 2 TEA TOWE
4970	False	False	False
4971	False	False	False
4972	False	False	False
4973	False	False	True
4974	False	False	False
4975	False	False	False
4976	False	False	False
4977	False	False	False
4978	False	False	False
4979	False	False	False

5. Tìm tập phổ biến bằng giải thuật apriori

```
# case 1: default minimum support = 0.5 does not use colnames (item names)
mod = mlxtend.frequent_patterns.apriori(online_encoder_df)

# case 2:
mod_minsupport =
    mlxtend.frequent_patterns.apriori( online_encode
    r_df,
    min_support=0.01
)
mod_minsupport.loc[0:6]
# case 3: add colnames for easier interpretability
mod_colnames_minsupport = mlxtend.frequent_patterns.apriori(
    online_encoder_df,
    min_support=0.01,
    use_colnames=True
)
mod_colnames_minsupport['length'] =
    ( mod_colnames_minsupport['itemsets'].apply(lambda x: len(x))
)
```

4.1.3. Bài tập trên lớp

Bài toán 2: Tìm tập phổ biến cho bài toán đề xuất phim. Dữ liệu lấy tại https://grouplens.org/datasets/movielens với MovieLens 100K dataset

Nhiệm vụ 1: Tìm tập phổ biến có trong tập dữ liệu phim

1. Import các gói thư viện, nạp dữ liệu và tiền xử lý dữ liệu vào notebook

```
import os
import pandas as pd
data_folder = "data/ml-100k"
ratings_filename = os.path.join(data_folder, "u.data")
all_ratings = pd.read_csv(ratings_filename, delimiter="\t", header=None,
```

SGU25

```
                    names = ["UserID", "MovieID", "Rating", "Datetime"])
all_ratings["Datetime"] = pd.to_datetime(all_ratings['Datetime'],
                                         unit='s')
# Tạo cột mới tên Favorable
all_ratings["Favorable"] = all_ratings["Rating"] > 3
ratings = all_ratings[all_ratings['UserID'].isin(range(200))]
favorable_ratings = ratings[ratings["Favorable"]]
favorable_reviews_by_users = dict((k, frozenset(v.values)) for k, v in
                        favorable_ratings.groupby("UserID")["MovieID"])
num_favorable_by_movie = ratings[["MovieID",
                        "Favorable"]].groupby("MovieID").sum()
num_favorable_by_movie.sort_values(by="Favorable",
ascending=False).head()
```

Kết quả thực hiện

5 rows 5 rows × 1 columns

MovieID	Favorable
50	100
100	89
258	83
181	79
174	74

2. Tạo hàm để tìm tập phổ biến

```
from collections import defaultdict
def find_frequent_itemsets(favorable_reviews_by_users, k_1_itemsets,
min_support):
    counts = defaultdict(int)
    for user, reviews in favorable_reviews_by_users.items():
        for itemset in k_1_itemsets:
            if itemset.issubset(reviews):
                for other_reviewed_movie in reviews - itemset:
                    current_superset = itemset |
                       frozenset((other_reviewed_movie,))
                    counts[current_superset] += 1
    return dict([(itemset, frequency) for itemset, frequency in
                   counts.items() if frequency >= min_support])
```

3. Tìm tập phổ biến

```
import sys
```

```
frequent_itemsets = {}  # itemsets are sorted by length
min_support = 50
# k=1 candidates are the isbns with more than min_support favourable
reviews
frequent_itemsets[1] = dict((frozenset((movie_id,)), row["Favorable"])
              for movie_id, row in num_favorable_by_movie.iterrows()
                    if row["Favorable"] > min_support)

print("There are {} movies with more than {} favorable
               reviews".format(len(frequent_itemsets[1]), min_support))

sys.stdout.flush()
for k in range(2, 20):
    # Generate candidates of length k, using the frequent itemsets of
length k-1
    # Only store the frequent itemsets
    cur_frequent_itemsets =
               find_frequent_itemsets(favorable_reviews_by_users,
                         frequent_itemsets[k-1], min_support)
    if len(cur_frequent_itemsets) == 0:
        print("Did not find any frequent itemsets of length
{}".format(k))
        sys.stdout.flush()
        break
    else:
        print("I found {} frequent itemsets of length
                   {}".format(len(cur_frequent_itemsets), k))
        #print(cur_frequent_itemsets)
        sys.stdout.flush()
        frequent_itemsets[k] = cur_frequent_itemsets
# We aren't interested in the itemsets of length 1, so remove those
del frequent_itemsets[1]
```

Kết quả thực hiện

```
There are 16 movies with more than 50 favorable reviews
I found 93 frequent itemsets of length 2
I found 295 frequent itemsets of length 3
I found 593 frequent itemsets of length 4
I found 785 frequent itemsets of length 5
I found 677 frequent itemsets of length 6
I found 373 frequent itemsets of length 7
I found 126 frequent itemsets of length 8
I found 24 frequent itemsets of length 9
I found 2 frequent itemsets of length 10
Did not find any frequent itemsets of length 11
```

SGU25

4.1.4. Bài tập về nhà

Tìm hiểu thư viện **apyori (https://pypi.org/project/apyori)** để thực hiện tìm tập phổ biến từ dữ liệu mua hàng tại siêu thị. Tham khảo bài hướng dẫn https://www.kaggle.com/code/rockystats/apriori-algorithm-or-market-basket-analysis

- Phần bài làm nằm trong thư mục có tên là "Code" và tên là "apyori_frequency_items.ipynb".

SGU25

4.2. XÁC ĐỊNH LUẬT KẾT HỢP TỪ TẬP PHỔ BIẾN

4.2.1. Ôn tập lý thuyết với GPT

1) Luật kết hợp (association rules) là gì? Hãy giải thích các khái niệm support, confidence, và lift trong luật kết hợp

- Luật kết hợp (Association Rules) là kỹ thuật khai phá dữ liệu để tìm ra các mối quan hệ thú vị giữa các sản phẩm/mục trong dữ liệu giao dịch. Ví dụ điển hình: {Nước ngọt} → {Bim bim} → Nghĩa là: khách mua nước ngọt thì rất hay mua thêm bim bim.

- Ba chỉ số quan trọng:

+ Support (Độ phổ biến): Tỷ lệ giao dịch chứa tập hợp mục đó trong toàn bộ dữ liệu.

$$\text{Support}(\{\text{Nước ngọt}\} \rightarrow \{\text{Bim bim}\}) = \%\text{ giao dịch có cả nước ngọt và bim bi}$$

→ Càng cao = quy tắc càng phổ biến, càng đáng quan tâm.

+ Confidence (Độ tin cậy): Khi đã mua X thì mua Y bao nhiêu % trường hợp?

$$\text{Confidence}(\{\text{Nước ngọt}\} \rightarrow \{\text{Bim bim}\}) = \text{Support(cả hai)} / \text{Support(Nước ngọt)}$$
$$= \%\text{ khách mua nước ngọt thì cũng mua bim bim}$$

→ Càng cao = quy tắc càng chắc chắn.

+ Lift (Độ nâng): Quy tắc tốt hơn bao nhiêu lần so với mua ngẫu nhiên.

$$Lift = Confidence/Support(Bimbim)$$

- Lift = 1 → không liên quan (mua độc lập)
- Lift > 1 → có liên kết tích cực (mua cái này kéo theo cái kia mạnh hơn ngẫu nhiên)
- Lift < 1 → liên kết tiêu cực (mua cái này làm giảm khả năng mua cái kia)

- Tóm tắt nhanh:

- Support cao → quy tắc phổ biến
- Confidence cao → quy tắc đáng tin
- Lift > 1 → quy tắc thực sự có ý nghĩa (không phải trùng hợp ngẫu nhiên)

2) Quá trình tìm luật kết hợp từ tập phổ biến bao gồm những bước nào? Làm thế nào để chuyển từ tập phổ biến sang luật kết hợp?

- Các bước chính:

+ Tìm tập phổ biến (Frequent Itemsets):

- Xác định ngưỡng support tối thiểu (min_support).
- Dùng thuật toán Apriori hoặc FP-Growth để tìm các tập mục có support ≥ min_support.

+ Sinh luật kết hợp từ tập phổ biến:

- Với mỗi tập phổ biến L có k phần tử (k ≥ 2), tạo tất cả các luật dạng X → Y sao cho:

$$X \cup Y = L$$

$$X \cap Y = \varnothing$$

- Ví dụ với tập {A, B, C}, các luật có thể là:

  {A,B} → {C}, {A,C} → {B}, {B,C} → {A}, {A} → {B,C}, {B} → {A,C}, {C} → {A,B}.

+ Lọc luật theo ngưỡng confidence:

- Tính confidence cho mỗi luật.
- Chỉ giữ lại luật có confidence ≥ min_confidence.

3) Khi nào một luật kết hợp được coi là "mạnh" (strong rule)? Các chỉ số như lift hoặc confidence ảnh hưởng như thế nào đến đánh giá này?

- Một luật kết hợp được coi là mạnh khi đồng thời thỏa mãn:

+ Support đủ cao (≥ min_support) — luật xuất hiện đủ thường xuyên.

+ Confidence đủ lớn (≥ min_confidence, thường ≥ 0.6 hoặc 60%) — luật có độ tin cậy cao.

+ Lift > 1 — có mối quan hệ dương giữa X và Y, tức là X làm tăng khả năng xuất hiện của Y.

- Ảnh hưởng của các chỉ số:

+ Confidence cao nhưng Lift thấp: Luật có thể không hữu ích vì Y vốn đã xuất hiện thường xuyên, không phải do X gây ra.

+ Lift cao nhưng Support thấp: Luật có thể không đáng tin cậy do ít dữ liệu hỗ trợ.

+ Luật lý tưởng: Support vừa phải + Confidence cao + Lift cao (> 1.5).

4) Viết đoạn code mẫu bằng Python (sử dụng thư viện như mlxtend hoặc apyori) để tìm luật kết hợp từ tập phổ biến không? Hãy mô tả các bước thực hiện

- Mã mẫu trong Python sử dụng thư viện mlxtend để tìm luật kết hợp từ tập phổ biến:

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
# Bước 1: Chuẩn bị dữ liệu giao dịch (danh sách các mặt hàng mỗi giao dịch)
transactions = [
    ['Sữa', 'Bánh mì', 'Bơ'],
    ['Bánh mì', 'Bơ'],
    ['Sữa', 'Bơ', 'Bia'],
    ['Sữa', 'Bánh mì', 'Bơ', 'Bia'],
    ['Sữa', 'Bánh mì'],
    ['Bánh mì', 'Bơ'],
    ['Sữa', 'Bơ'],
    ['Sữa', 'Bánh mì', 'Bơ', 'Bia'],
    ['Sữa', 'Bánh mì', 'Bơ'],
    ['Bia', 'Trứng']
]
# Bước 2: Chuyển dữ liệu sang dạng one-hot encoding
te = TransactionEncoder()
te_data = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_data, columns=te.columns_)
# Bước 3: Tìm tập phổ biến với min_support = 0.3 (30%)
min_support = 0.3
frequent_itemsets = apriori(df, min_support=min_support, use_colnames=True)
# Bước 4: Sinh luật kết hợp với min_confidence = 0.6 (60%)
min_confidence = 0.6
rules = association_rules(frequent_itemsets, metric="confidence",
min_threshold=min_confidence)
# Bước 5: Lọc luật mạnh (lift > 1)
```

SGU25

```
strong_rules = rules[rules['lift'] > 1.0].sort_values('lift', ascending=False)
# In các luật mạnh
for idx, rule in strong_rules.iterrows():
    antecedents = ', '.join(list(rule['antecedents']))
    consequents = ', '.join(list(rule['consequents']))
    print(f"Luật: {antecedents} → {consequents}")
    print(f"  Support:    {rule['support']:.3f}")
    print(f"  Confidence: {rule['confidence']:.3f}")
    print(f"  Lift:       {rule['lift']:.3f}\n")
```

- Mô tả các bước thực hiện trong code:

+ Bước 1: Chuẩn bị dữ liệu giao dịch dưới dạng danh sách các mặt hàng mỗi giao dịch.

+ Bước 2: Chuyển đổi dữ liệu sang dạng ma trận nhị phân (one-hot encoding) để thuật toán có thể xử lý.

+ Bước 3: Tìm các tập mục phổ biến (frequent itemsets) bằng thuật toán Apriori với ngưỡng support tối thiểu.

+ Bước 4: Sinh các luật kết hợp từ tập phổ biến, tính toán các chỉ số support, confidence, lift; lọc theo ngưỡng confidence.

+ Bước 5: Lọc ra các luật mạnh dựa trên điều kiện lift > 1 và in ra kết quả.

4.2.2. Bài làm mẫu

Bài toán 1: Xác định các luật kết hợp có trong tập dữ liệu giao dịch mua hàng tại siêu thị. Dữ liệu lấy từ https://www.kaggle.com/code/mgmarques/customer-segmentation-and-market-basket-analysis

Nhiệm vụ 1: Xác định các luật kết hợp

1. Thực hiện các bước từ 1 đến 4 và chọn 1 trong 3 case của bước 5 để tìm tập phổ biến có trong dữ liệu giao dịch dạng bảng của siêu thị

2. Trích các luật có trong tập phổ biến đã thực hiện ở bước 1

```
# case 1: hiển thị 6 luật trong 1064 luật được tìm thấy
rules =
    mlxtend.frequent_patterns.association_rules( mod_colnames_minsupp
    ort,
    metric="confidence",
    min_threshold=0.6,
    support_only=False
)
rules.loc[0:6]
# case 2: liệt kê các rule thỏa mãn điều kiện min_threshold
rules2 =
    mlxtend.frequent_patterns.association_rules( mod_colnames_minsupp
    ort,
    metric="lift", #có thể sử dụng support, confidence, …
    min_threshold=50,
    support_only=False
```

SGU25

```
print("Number of Associations: {}".format(rules2.shape[0])) # 170 luật
```

Kết quả thực hiện case 1

7 rows | 7 rows × 10 columns

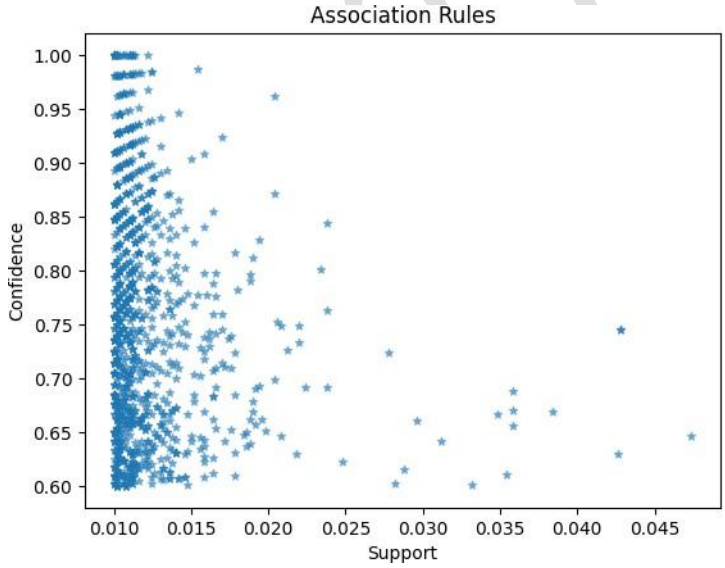
	antecedents	consequents	antecedent support	consequent sup...	support	confidence	lift
1	(ALARM CLOCK BAKELIKE CHOCOLATE)	(ALARM CLOCK BAKELIKE GREEN)	0.0208	0.0546	0.0140	0.673077	12.327416
2	(ALARM CLOCK BAKELIKE CHOCOLATE)	(ALARM CLOCK BAKELIKE RED)	0.0208	0.0520	0.0140	0.673077	12.943787
0	(12 PENCILS SMALL TUBE SKULL)	(12 PENCILS SMALL TUBE RED RETROSPOT)	0.0222	0.0276	0.0152	0.684685	24.807416
3	(ALARM CLOCK BAKELIKE IVORY)	(ALARM CLOCK BAKELIKE GREEN)	0.0268	0.0546	0.0164	0.611940	11.207698
4	(ALARM CLOCK BAKELIKE ORANGE)	(ALARM CLOCK BAKELIKE GREEN)	0.0278	0.0546	0.0208	0.748201	13.703323
5	(ALARM CLOCK BAKELIKE RED)	(ALARM CLOCK BAKELIKE GREEN)	0.0520	0.0546	0.0358	0.688462	12.609186
6	(ALARM CLOCK BAKELIKE GREEN)	(ALARM CLOCK BAKELIKE RED)	0.0546	0.0520	0.0358	0.655678	12.609186

Hình 4.1 – Trình bày 6 luật kết hợp

3. Hình vẽ cho thấy mối quan hệ giữa support, confidence của luật

```
rules.plot.scatter("support", "confidence", alpha=0.5, marker="*")
plt.xlabel("Support")
plt.ylabel("Confidence")
plt.title("Association Rules")
plt.show()
```

Kết quả thực hiện

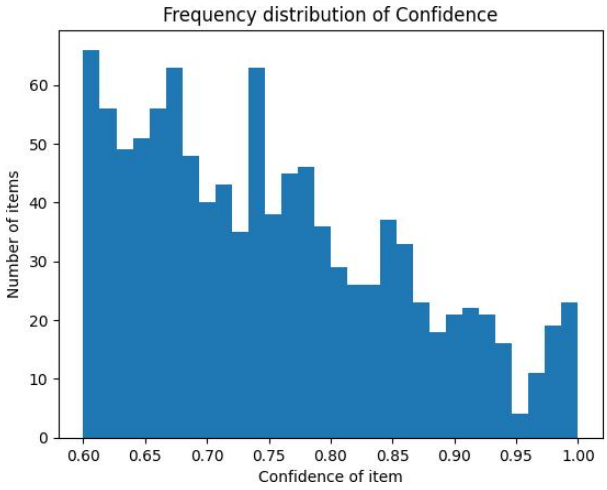


Hình 4.2 - Biểu đồ mối quan hệ giữa support và confidence có trong mối kết hợp

4. Hình vẽ cho thấy mối quan hệ giữa số item và confidence của luật

```
rules.hist("confidence", grid=False, bins=30)
plt.xlabel("Confidence of item")
plt.ylabel("Number of items")
plt.title("Frequency distribution of Confidence")
plt.show()
```

Kết quả thực hiện



4.2.3. Bài tập trên lớp

Bài toán 2: Tìm luật kết hợp trong tập dữ liệu phim. Dữ liệu lấy tại https://grouplens.org/datasets/movielens với MovieLens 100K dataset

Nhiệm vụ 1: Tìm các luật kết hợp từ tập phổ biến

1. Thực hiện các bước trong bài tập 1 để tìm tập phổ biến có trong dữ liệu đề xuất phim.

   - Xem lại nhiệm vụ 1 ở mục 4.1.3 để có thể biết cách tìm tập phổ biến có trong dữ liệu đề xuất phim.

2. Xác định danh sách các tập luật ứng tuyển

```
# Now we create the association rules. First, they are candidates until
the confidence has been tested
candidate_rules = []
for itemset_length, itemset_counts in frequent_itemsets.items():
    for itemset in itemset_counts.keys():
        for conclusion in itemset:
            premise = itemset - set((conclusion,))
            candidate_rules.append((premise, conclusion))
# There are 15285 candidate rules
print("There are {} candidate rules".format(len(candidate_rules)))
```

3. Tính mức độ tin cậy (confidence) từng luật ứng tuyển

```
# Now, we compute the confidence of each of these rules. This is very
similar to what we did in chapter 1
correct_counts = defaultdict(int)
incorrect_counts = defaultdict(int)
for user, reviews in favorable_reviews_by_users.items():
    for candidate_rule in candidate_rules:
        premise, conclusion = candidate_rule
        if premise.issubset(reviews):
            if conclusion in reviews:
                correct_counts[candidate_rule] += 1
            else:
```

SGU25

```
                incorrect_counts[candidate_rule] += 1
rule_confidence = {candidate_rule: correct_counts[candidate_rule] /
float(correct_counts[candidate_rule] + incorrect_counts[candidate_rule])
              for candidate_rule in candidate_rules}
```

4. Lấy các luật có độ tin cậy > 0.9

```
min_confidence = 0.9
# Filter out the rules with poor confidence
rule_confidence = {rule: confidence for rule, confidence in
                rule_confidence.items() if confidence > min_confidence}
print(len(rule_confidence)) #5152 luật
```

5. Liệt kê năm luật kết hợp có độ tin cậy cao nhất

```
from operator import itemgetter
sorted_confidence = sorted(rule_confidence.items(), key=itemgetter(1),
reverse=True)
for index in range(5):
    print("Rule #{0}".format(index + 1))
    (premise, conclusion) = sorted_confidence[index][0]
    print("Rule: If a person recommends {0} they will also recommend
                        {1}".format(premise, conclusion))
    print(" - Confidence: {0:.3f}".format(rule_confidence[(premise,
                                conclusion)]))
    print("")
```

Kết quả thực hiện

```
Rule #1
Rule: If a person recommends frozenset({98, 181}) they will also
recommend 50
 - Confidence: 1.000

Rule #2
Rule: If a person recommends frozenset({172, 79}) they will also
recommend 174
 - Confidence: 1.000

Rule #3
Rule: If a person recommends frozenset({258, 172}) they will also
recommend 174
 - Confidence: 1.000
```

Hình 4.3 - Các luật kết hợp trình bày dạng văn bản

6. Hiển thị tên phim cụ thể trong các luật kết hợp đã tìm thấy

```
# we can get the movie titles themselves from the dataset
movie_name_filename = os.path.join(data_folder, "u.item")
movie_name_data = pd.read_csv(movie_name_filename, delimiter="|",
                    header=None, encoding = "mac-roman")
```

```
movie_name_data.columns = ["MovieID", "Title", "Release Date", "Video
                 Release", "IMDB", "<UNK>", "Action", "Adventure",
                "Animation", "Children's", "Comedy", "Crime",
                "Documentary", "Drama", "Fantasy", "Film-Noir",
                "Horror", "Musical", "Mystery", "Romance",
                "Sci-Fi", "Thriller", "War", "Western"]
def get_movie_name(movie_id):
    title_object = movie_name_data[movie_name_data["MovieID"] ==
                                movie_id]["Title"]
    title = title_object.values[0]
    return title

for index in range(5):
    print("Rule #{0}".format(index + 1))
        (premise, conclusion) = sorted_confidence[index][0]
    premise_names = ", ".join(get_movie_name(idx) for idx in premise)
    conclusion_name = get_movie_name(conclusion)
    print("Rule: If a person recommends {0} they will also recommend
            {1}".format(premise_names, conclusion_name))
    print(" - Confidence: {0:.3f}".format(rule_confidence[(premise,
            conclusion)]))
    print("")
```

Kết quả thực hiện

```
Rule #1
Rule: If a person recommends Silence of the Lambs, The (1991), Return of
 the Jedi (1983) they will also recommend Star Wars (1977)
 - Confidence: 1.000

Rule #2
Rule: If a person recommends Empire Strikes Back, The (1980), Fugitive,
The (1993) they will also recommend Raiders of the Lost Ark (1981)
 - Confidence: 1.000
```

4.2.4. Bài tập về nhà

Tìm hiểu thư viện **apyori (https://pypi.org/project/apyori)** để thực hiện tìm luật kết hợp từ tập phổ biến từ dữ liệu mua hàng tại siêu thị. Tham khảo bài hướng dẫn https://www.kaggle.com/code/rockystats/apriori-algorithm-or-market-basket-analysis

- Phần bài làm nằm trong thư mục có tên là "Code" và tên là "apyori_association_rules.ipynb".

D. TÓM TẮT CHƯƠNG

Với 2 phần là xác định tập phổ biến với giải thuật Apriori và xác định luật kết hợp được trình bày trong chương đã phần nào giúp sinh viên có thể tiến hình triển khai tìm tập phổ biến và các luật kết hợp trên một số tập dữ liệu cơ bản.

Với việc thực hành giải quyết các bài tập trên lớp và các bài tập ở nhà giúp sinh viên hiểu rõ hơn các khái niệm lý thuyết đã học và áp dụng tốt vào việc giải quyết các bài toán thực tế.

SGU25